

Chapter 5: Web Application Security

By Er. Ukesh Thapa

Syllabus

5.1 Common vulnerabilities: XSS, SQL injection, CSRF

5.2 Security best practices: Input validation, sanitization, https, secure cookies, env variables

5.3 Authentication practices and token handling

5.4 Security in full-stack apps: CORS, safe sessions

Total Marks: 8

Web Security

- ❖ Web security refers to the practices and technologies used to protect **websites, web applications, and data** from **cyber threats and unauthorized access**.
 - Protects data during transmission over the Internet.
 - Safeguards websites, servers, users, and applications.
 - Prevents data breaches, malware attacks, and hacking.
 - Essential due to the increasing number of cyberattacks.

Web Security

❖ CIA Triad

- **Confidentiality:** Ensuring only authorized users can see the data.
- **Integrity:** Ensuring the data hasn't been tampered with or changed.
- **Availability:** Ensuring the website stays up and running for legitimate users.

Layer of Defense

❖ Network Security

- This involves protecting the "pipes" that carry the data.
 - **HTTPS/TLS: Encrypting the connection** between the user's **browser** and your **server** so "eavesdroppers" can't read the traffic.
 - **Firewalls (WAF):** A Web Application Firewall filters out **malicious traffic** before it even reaches your code.

Layer of Defense

❖ **Application Security (The Code)**

- This is where developers focus. It involves writing code that handles data safely.
 - **Input Validation:** Never trusting what a user types into a form.
 - **Authentication:** Making sure users are who they say they are (**MFA, strong passwords**).
 - **Authorization:** Making sure a regular user can't access an admin panel.

Practices for Securing Website

- ❖ Keep Software Updated
- ❖ Beware of SQL Injection
- ❖ Prevent Cross-Site Scripting (XSS)
- ❖ Limit Error Messages
- ❖ Validate User Input
- ❖ Use Strong Passwords
- ❖ Implement HTTPS:
- ❖ Enable Two-Factor Authentication (2FA)
- ❖ Access Control
- ❖ Monitor and Log Activity
- ❖ Use Modern and Secure Tech Stacks

Major Cyber Threats

- ❖ Ransomware
- ❖ SQL Injection
- ❖ Phishing
- ❖ Viruses and worms
- ❖ Spyware
- ❖ Cross-site scripting (XSS)
- ❖ Code Injection
- ❖ Denial of Service

Note: for latest cyber threats in recent years → <https://owasp.org/Top10/2025/>

Web Security Tools for Developers and Security Experts

- ❖ **OWASP ZAP:** A free, open-source tool used to find **security flaws in web applications**.
- ❖ **Burp Suite:** A powerful platform for **analyzing and testing** web application security.
- ❖ **Cloudflare:** Offers protection **against DDoS attacks** and enhances **performance** with **CDN services**.
- ❖ **SSL Labs:** Evaluates the **security of your SSL/TLS setup** and provides a detailed **report**.
- ❖ **Snyk:** Detects vulnerabilities in **open-source libraries** and helps fix them quickly.

SQL Injection (SQLi)

- ❖ SQL Injection happens when an attacker inserts malicious SQL fragments into an input field.
- ❖ The backend server unknowingly concatenates this input into a database query, changing the query's logic.
- ❖ Demonstration
 - `query = "SELECT * FROM users WHERE username = " + user_input + ";"`
 - Normal Input: admin → Result: `SELECT * FROM users WHERE username = 'admin';`
 - Malicious Input: `' OR '1'='1`
 - The Resulting Query:
 - `SELECT * FROM users WHERE username = " OR '1'='1';`
 - Since `'1'='1'` is always true, the database returns all users, often logging the attacker in as the first user (usually the admin) without a password.

SQL Injection (SQLi)

- ❖ **Impact:** Data theft, unauthorized access, and the ability to delete entire tables.
- ❖ **Defensive Measure (DM):** Use Parameterized Queries (Prepared Statements). This treats user input as literal text, not executable code.
 - **Correct Code:** `db.execute("SELECT * FROM users WHERE username = ?", [user_input])`

Cross-Site Scripting (XSS)

- ❖ XSS occurs when an application includes untrusted data in a web page without proper validation.
- ❖ This allows an **attacker to execute malicious JavaScript** in the victim's browser.
- ❖ Cross-Site Scripting (XSS) attacks occur when:
 - Data enters a Web application through an untrusted source, most frequently a web request.
 - The data is included in dynamic content that is sent to a web user without being validated for malicious content.

Stored XSS Attacks

- The **attacker uploads a malicious script directly to the website's database** (for example, in a comment section, a user profile bio, or a forum post).
- The script sits quietly on the server. The attacker doesn't need to send a malicious link to anyone; they just wait for users to visit the page.
- Whenever any user views the page where the "comment" or "bio" is displayed, the website serves the malicious script as part of the page.
- A single attack can infect thousands of users. Every person who loads that page becomes a victim because their browser thinks the script is a legitimate part of the site.
- The Trust Trap: Since the code is coming directly from the website's own database, browser security filters are less likely to block it compared to a suspicious URL.

Reflected XSS Attacks

- The attack happens when a **website takes input from a URL** and displays it back on the screen without checking it first (like a **search result or an error message**).
- The attacker doesn't hack the website's database. Instead, they **hide a piece of malicious JavaScript inside a link and trick a user into clicking it**.
- The **Trust Trap**: Because the script appears to come from the "trusted" website (like **your bank or social media**), the browser executes the code automatically.
- **One-Time Attack**: It is non-persistent. The script isn't saved on the site; it only **runs for the specific person** who clicked that one malicious link.

Blind XSS Attacks

- The attacker submits a **malicious script** into a part of the site that is meant for internal staff, such as **feedback forms, support tickets, or "Contact Us" pages**.
- **Persistent Storage:** The website **saves the script in its database**. It doesn't trigger on the public-facing site, so everything looks normal to the attacker.
- **The Delayed Trigger:** The attack only "wakes up" when a different user—usually a system administrator or support agent—logs into a private dashboard to view that specific entry.
- **Administrative Access:** This is especially dangerous because the victim is often an **Admin**. If the script steals a **session cookie**, the attacker gains full control over the entire website's backend.
- **Hard to Detect:** Since the "reflection" happens on a private page the attacker can't see, they use tools like **XSS Hunter**. These tools send a "**ping**" back to the **attacker** as soon as the **script is viewed by an admin, telling them exactly where and when the attack worked**.

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Security best practices: Input Validation

- ❖ Use a **whitelist** (Allow-listing) approach to define exactly what 'good' data looks like.
- ❖ Validate **data types** (string, integer, boolean) and ensure they match expectations.
- ❖ Check **data range and length** (e.g., age 0-120, username < 20 characters).
- ❖ Use **Regex** for format validation (emails, zip codes, phone numbers).
- ❖ Enforce **Syntactic** Validation to ensure correct structure (e.g., valid JSON).
- ❖ Enforce **Semantic** Validation to ensure data makes sense in context (e.g., sufficient account balance).
- ❖ Implement **Multi-Layer Validation** on both the Client-Side (for UX) and Server-Side (for security).
- ❖ Apply **Database Constraints** as a final safety net (NOT NULL, CHECK, Foreign Keys).
- ❖ **Normalize/Canonicalize** input to handle encoding tricks and prevent Path Traversal.
- ❖ Use **Parameterized Queries** for all database interactions to prevent SQL Injection.

Security best practices: Sanitization

- ❖ Sanitize data based on where it will be placed (e.g., HTML body, HTML attributes, JavaScript, or CSS) to prevent "breaking out" of the intended container.
- ❖ Convert potentially dangerous characters like < to <, > to >, and & to & before rendering them in the browser.
- ❖ Use a **Whitelist for Tags**: When allowing "Rich Text" (like in a blog editor), only permit a strict list of safe tags (e.g., , <i>, <a>) and reject everything else.
- ❖ Remove **dangerous event-handler** attributes such as **onload**, **onerror**, or **onclick** from user-provided HTML.
- ❖ If allowing user-submitted links, ensure they start with **http://** or **https://** and strip out javascript: or data: protocols.

Security best practices: Sanitization

- ❖ Remove **navigation shortcuts** like ../ or ./ from file paths to prevent Path Traversal attacks that could expose sensitive system files.
- ❖ Strip **null characters** (e.g., %00) that attackers use to bypass file extension filters or terminate strings prematurely.
- ❖ **Store "raw" validated data** in your database, but sanitize it at the moment of display. This ensures the data remains flexible for different platforms (web, mobile, email).
- ❖ Leverage Trusted Libraries: Do not use custom Regex for sanitization. Use battle-tested libraries like **DOMPurify (JS)**, **Bleach (Python)**, or **HtmlSanitizer (.NET)**.
- ❖ **Parameterized Queries (SQL)**: Use database drivers to treat input as literal data, effectively "sanitizing" it against SQL Injection.

Security best practices: https

- ❖ **Use Strong Cipher Suites:** Configure your server to use **strong encryption algorithms** (like AES-GCM) and disable weak ones (like RC4 or DES).
- ❖ **Enable Perfect Forward Secrecy (PFS):** Ensure that if your server's private key is compromised in the future, past recorded encrypted traffic cannot be decrypted.
- ❖ **Automate Certificate Renewal:** Use tools like **Certbot (Let's Encrypt)** to **automatically renew certificates before they expire**, avoiding site downtime and "Your connection is not private" errors.
- ❖ **CAA (Certification Authority Authorization):** Add a **CAA record to your DNS** to specify exactly **which Certificate Authorities** (like DigiCert or Let's Encrypt) are **allowed to issue certificates** for your domain.
- ❖ **OCSP Stapling:** Enable this on your server to improve **performance and privacy** by providing the **certificate's revocation status directly to the user's browser**.

Security best practices: https

- ❖ **Enforce Full-Site HTTPS:** Do not just secure the **login page**; encrypt every single page and **asset (images, CSS, JS)** to prevent "Mixed Content" warnings and session hijacking.
- ❖ **Use TLS 1.3 (or 1.2):** Disable outdated and vulnerable protocols like SSL 2.0, SSL 3.0, TLS 1.0, and TLS 1.1. TLS 1.3 is the current gold standard for speed and security.
- ❖ **Implement HSTS (HTTP Strict Transport Security):** Use this response header to tell browsers to only communicate with your site via HTTPS, preventing "protocol downgrade" attacks.
- ❖ **Redirect HTTP to HTTPS:** Automatically redirect all port 80 (HTTP) traffic to port 443 (HTTPS) using a 301 Moved Permanently status code.
- ❖ **Secure Cookie Flag:** Ensure all session cookies are marked with the Secure attribute so the browser never sends them over an unencrypted connection.

Security best practices: Secure Cookies

- ❖ **HttpOnly Flag:** Prevents client-side scripts (JavaScript) from accessing the cookie. This is the most effective defense against Session Hijacking via XSS.
- ❖ **Secure Flag:** Ensures the cookie is only transmitted over **encrypted HTTPS** connections. It will **never be sent over plain HTTP**, preventing **"Man-in-the-Middle" sniffing**.
- ❖ **SameSite Attribute:** Controls whether cookies are **sent with cross-site requests**.
- ❖ **Strict:** Cookie is only sent for **requests originating from the same site**.
- ❖ **Lax:** (Standard) Provides a balance; the cookie is sent during top-level navigations (like clicking a link).
- ❖ **None:** Requires the **Secure flag and allows cross-site sharing** (only use if necessary).

Security best practices: Secure Cookies

- ❖ **Cookie Prefixing:** Use prefixes like **__Host-** or **__Secure-** in the cookie name. These are enforced by the browser to ensure the cookie was set securely and from the correct domain.
- ❖ **Short Expiration (Max-Age / Expires):** Avoid "infinite" sessions. Set a reasonable expiration time so that stolen cookies have a limited window of use.
- ❖ **Session Invalidation:** Ensure that when a user **logs out**, the **cookie is deleted** both on the client side (via an expired header) and on the server side (invalidating the session ID).
- ❖ **Minimize Data Storage:** Never store sensitive info (passwords, PII) directly in a cookie. Only **store** a randomly generated **Session ID** or a **cryptographically** signed token.
- ❖ **Domain and Path Scoping:** **Restrict** the cookie to the **most specific domain and path possible (e.g., path=/api)** to prevent it from being sent to unnecessary parts of your application

Password Hashing

- ❖ Don't try to write a hashing function using math. **Use a trusted library** (e.g., bcryptjs for Node, passlib for Python).
- ❖ **Adaptive Hashing:** As computers get faster, you should increase your "Work Factor." bcrypt and Argon2 allow you to change this without changing your code structure.
- ❖ **Validation, Not Decryption:** You **cannot "decrypt" a hash**. To check a password, you hash the attempt and compare it to the stored hash.
- ❖ **On-Login Re-hashing:** If you upgrade from bcrypt to Argon2, re-hash the user's password the **next time they log in** so their **security is automatically updated**.

Types of Hashing Algorithms

- ❖ **Argon2:** The **current Gold Standard** (winner of the Password Hashing Competition). It allows you to configure memory usage, time, and parallelism, making it **resistant to GPU/ASIC cracking**.
- ❖ **bcrypt:** The Industry Standard. It has been around since 1999 and is still incredibly secure because it is "intentionally slow." It uses a "work factor" to increase security over time.
- ❖ **scrypt:** Designed to be **memory-intensive**, making it very **expensive for hackers** to build custom hardware (ASICs) to crack it.
- ❖ **PBKDF2:** A standard (often used in government/enterprise) that applies a hash function thousands of times. It is secure but generally considered less resistant to GPU attacks than Argon2.
- ❖ **Legacy (Avoid): MD5, SHA-1, and SHA-256.** Explain that these are fast cryptographic hashes. They are **meant for files, not passwords**. A hacker can test billions of MD5 hashes per second.

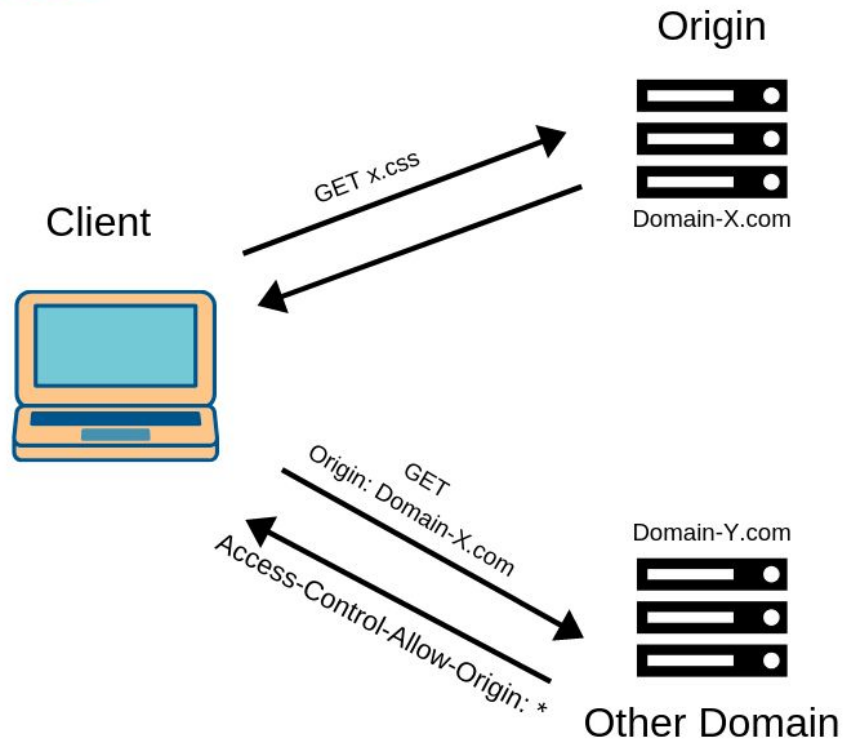
Secure Token Handling

- ❖ Use HttpOnly & Secure Cookies: This makes tokens inaccessible to JavaScript, effectively killing XSS-based token theft.
- ❖ Use SameSite=Lax or Strict: This tells the browser to only send the cookie to your domain, preventing CSRF.
- ❖ Do NOT use LocalStorage/SessionStorage: Any script on your page (including third-party ads or libraries) can read these.
- ❖ Keep Access Tokens short-lived: Set them to expire in 5–15 minutes. If stolen, they become useless quickly.
- ❖ Use Refresh Tokens for long sessions: Store these in your database so you can revoke them if a user logs out or is compromised.
- ❖ JWTs are encoded, not encrypted. Never put passwords, emails, or PII inside them.
 - Is it a cookie? (Yes = Good, No = Check for XSS)
 - Is it HttpOnly? (Yes = Safe from JS)
 - Is it short-lived? (Yes = Limited damage if stolen)
 - Is the secret in an Env Var? (Yes = Safe from GitHub leaks)

Cross-Origin Resource Sharing

- ❖ An HTTP-header-based security mechanism that **allows a server to authorize web applications from different domains to access its resources**, bypassing the default browser Same-Origin Policy (SOP).
- ❖ **CORS Policy:**
 - Protects users by preventing malicious sites from reading sensitive data on another domain.
 - Browsers send an OPTIONS (pre-flight) request to check if the server allows the actual request.
 - **Allowed Origins:** Defines which domains are authorized to access the API (e.g., specific domains or * for all).
 - **Allowed Methods/Headers:** Restricts types of HTTP requests (GET, POST) and custom headers allowed in cross-origin requests.
 - **Credentials:** Controls whether **cookies, authorization headers, or TLS certificates are sent**.
- ❖ **Commonly Used CORS HTTP Headers:**
 - **Access-Control-Allow-Origin:** Specifies which origins (domain) can access the resource.
 - **Access-Control-Allow-Methods:** Lists allowed HTTP methods (e.g., GET, POST, PUT).
 - **Access-Control-Allow-Headers:** Lists headers that can be used during the request.
 - **Access-Control-Max-Age:** Caches the pre-flight request response for a set duration.

CORS



Session Hardening

- ❖ Session hardening involves securing **user sessions against hijacking, fixation, and unauthorized access** by implementing **short session timeouts, regenerating session IDs upon authentication**, using secure cookie attributes (HttpOnly, Secure, SameSite), and validating user behavior to detect anomalies.
- ❖ It is a critical component of application security.
- ❖ Key techniques for session hardening:
 - **Session Management:**
 - Implement short idle timeouts (e.g., 15-30 minutes) and absolute session lifetimes.
 - Use long, random, and complex session tokens. Always use HttpOnly, Secure, and SameSite flags for cookies,

Session Hardening



➤ **Behavioral Monitoring:**

- Monitor for unexpected behavior like new devices, unfamiliar locations.
- Implement step-up authentication (like MFA) when high-risk actions are detected.

➤ **Protocol & Infrastructure Security:**

- Ensure TLS session resumption is secure, for example, by using short-lived session ticket keys to prevent long-term session hijacking.
- Browser Security: On endpoints, follow guidelines like CIS Benchmarks for browsers, which include disabling unnecessary features like developer tools.
- RDS Security: When using Remote Desktop Services, harden the host environment to restrict access, as users may otherwise bypass application controls.

Browser Security Settings.

❖ **Content Security Policy (CSP)**

- which sources of **content (scripts, styles, images)** are trusted.
- Use a strict policy like `default-src 'self';`. This blocks any script not hosted on your own domain.
- CSP **prevents "Inline Scripts"** (scripts written directly in HTML) and `eval()`, which are common hacker entry points.

❖ **HTTP Strict Transport Security (HSTS)**

- HSTS strengthens the transition from HTTP to HTTPS.
- It informs the browser that the site should only be accessed using HTTPS.
- Set a long max-age (e.g., one year) and include the `includeSubDomains` directive.
- prevents SSL Stripping attacks, where a hacker tries to "downgrade" the connection to unencrypted HTTP.

Browser Security Settings.

❖ Clickjacking Protection

- It prevents Clickjacking, where an **attacker** overlays your site with an **invisible layer to trick users** into clicking buttons (like "Delete Account").
- Use DENY (no one can frame you) or SAMEORIGIN (only you can frame yourself).

❖ Referrer-Policy

- Controls **how much information is sent in the Referer header** when a user clicks a link that leaves your site.
- URLs contain sensitive data (like reset-password?token=123). If a user clicks an external link, that token is sent to the new site.

❖ Permissions-Policy

- Allows the site to disable specific browser features for security and privacy.
- Explicitly disable the camera, microphone, and geolocation if your app doesn't need them (e.g., camera=(), microphone=()). This ensures that even if a script is compromised, it can't turn on the user's webcam.